

enter the following into the interactive shell to write the text “Hello, world!” to a file named *spam.txt*:

```
>>> # Unpythonic Example
>>> fileObj = open('spam.txt', 'w')
>>> fileObj.write('Hello, world!')
13
>>> fileObj.close()
```

Writing code this way can lead to unclosed files if, say, an error occurs in a `try` block and the program skips the call to `close()`. For example:

```
>>> # Unpythonic Example
>>> try:
...     fileObj = open('spam.txt', 'w')
...     eggs = 42 / 0    # A zero divide error happens here.
...     fileObj.close() # This line never runs.
... except:
...     print('Some error occurred.')
...
Some error occurred.
```

Upon reaching the zero divide error, the execution moves to the `except` block, skipping the `close()` call and leaving the file open. This can lead to file corruption bugs later that are hard to trace back to the `try` block.

Instead, you can use the `with` statement to automatically call `close()` when the execution leaves the `with` statement’s block. The following pythonic example does the same task as the first example in this section:

```
>>> # Pythonic Example
>>> with open('spam.txt', 'w') as fileObj:
...     fileObj.write('Hello, world!')
...
```

Even though there’s no explicit call to `close()`, the `with` statement will know to call it when the execution leaves the block. .

Use `is` to Compare with `None` Instead of `==`